

SIMATS ENGINEERING

CO2_ASSESSMENT TOOL2_ Scenario Based Assignment

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192472208
Name	Venganapalli Sireesha

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:50

Code of Conduct:

I V.Sireesha(192472208) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V.Sireesha

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario:** A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario	10
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues	10
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts	12.5

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided	10
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand	7.5
				Total	50

1.(i) Greedy Best-First Search (GBFS)

Greedy Best-First Search selects the path with the lowest heuristic value (straight-line distance) to the destination. In the flood scenario, the drone always chooses the route that appears closest to the hospital or delivery point without considering the actual travel cost.

Strengths

- Finds a route quickly.
- Requires less computation.
- Suitable for emergency situations where a fast decision is required.

Weaknesses

- Ignores actual travel cost such as floods, bad weather, and difficult terrain.
- May choose blocked or unsafe routes.
- Does not guarantee the shortest or safest path.
- Performance decreases when heuristic information is inaccurate.

(ii) A* Search

A* Search evaluates each path using

$$f(n) = g(n) + h(n)$$

where:

- $g(n)$ = actual travel cost from the start.
- $h(n)$ = estimated distance to the goal.

The drone considers both the distance and the actual conditions such as weather, flooded roads, and terrain before selecting the next path.

Advantages

- Finds the optimal path when the heuristic is admissible.
- Avoids expensive or risky routes.
- Produces safer and more reliable navigation.
- Balances speed and path quality.

(iii) Impact of Heuristic Accuracy

Greedy Best-First Search

- Highly dependent on heuristic accuracy.
- Accurate heuristic → faster search.
- Poor heuristic → wrong decisions and unsafe routes.

A*

- Uses both actual cost and heuristic.
- Accurate heuristic improves speed.
- Even with small heuristic errors, A* usually finds a near-optimal solution.

Therefore, heuristic accuracy affects GBFS more than A*.

(iv) Effect of Dynamic Changes

Blocked roads and changing weather require the drone to update its route.

GBFS

- May continue toward blocked paths.
- Often needs to restart the search.
- Less efficient in dynamic environments.

A*

- Recalculates path using updated costs.
- Chooses alternative safe routes.
- Handles changing environments more effectively.

(v) Recommended Algorithm

Recommended Algorithm: A*

Justification

- Considers both actual travel cost and estimated distance.
- Produces optimal and safe routes.
- Adapts better to blocked roads and weather changes.
- Reduces delivery delay while maintaining safety.
- More reliable than Greedy Best-First Search for emergency medicine delivery.

2.(i) Explain how Hill Climbing works and identify its limitations

Hill Climbing starts with an initial traffic signal plan and makes small changes to improve traffic flow. It always selects the neighboring solution that provides the greatest improvement.

Advantages

- Simple and easy to implement.
- Requires less memory.
- Quickly finds better solutions.

Limitations

- Can get stuck at a local optimum.
- Cannot guarantee the best global solution.
- Performance depends on the starting solution.
- Not suitable for highly dynamic traffic conditions.

(ii) Discuss local maxima, plateaus, and ridges with real-world interpretation

Local Maxima

- The algorithm reaches a good solution but not the best one.
- *Example:* Traffic improves in one area while congestion remains high elsewhere.

Plateau

- Many neighboring solutions have the same value, so the algorithm cannot determine which direction to move.
- *Example:* Several signal timing changes produce the same average waiting time.

Ridge

- The best solution requires multiple coordinated changes, but Hill Climbing changes only one step at a time.
- *Example:* Reducing congestion requires adjusting several nearby intersections together rather than one intersection alone.

(iii) Explain how Simulated Annealing or Genetic Algorithms overcome these limitations

Simulated Annealing

- Sometimes accepts worse solutions temporarily.
- Escapes local maxima.
- Explores different regions of the search space.
- Has a better chance of finding the global optimum.

Genetic Algorithm

- Maintains multiple candidate solutions.
- Uses selection, crossover, and mutation to generate better traffic signal plans.
- Explores many possible solutions simultaneously.
- Adapts well to changing traffic conditions.

(iv) Recommend the best approach and justify your choice

Recommended Approach: Genetic Algorithm

Justification

- Handles very large search spaces efficiently.
- Avoids local maxima by exploring multiple solutions.
- Can optimize several objectives at the same time, such as waiting time, fuel consumption, and congestion.
- Adapts well to changing traffic patterns.
- Scales effectively for a smart city with hundreds of intersections.

3. (i) Explain why this problem requires an Online Search Agent instead of Offline Search

An Online Search Agent is required because the rover does not have a complete map of the Martian surface before starting its mission. It must explore the environment, collect information, and make decisions while moving.

An Offline Search requires complete knowledge of the environment before planning a path, which is not possible on Mars.

Advantages of Online Search

- Works with unknown environments.
- Makes decisions in real time.
- Updates its path as new information becomes available.
- Safely explores unexplored terrain.

(ii) Analyze the challenges of partial observability and dynamic environments

Partial Observability

- The rover can sense only nearby areas.
- Hidden rocks or craters may not be detected until it gets closer.
- Limited sensor range increases uncertainty.

Dynamic Environment

- Dust storms or changing terrain may block the planned path.
- New obstacles can appear during exploration.
- The rover must continuously update its decisions to ensure safety.

These challenges require the rover to adapt its path based on newly discovered information.

(iii) Describe how the rover builds and updates its internal model

The rover creates an internal map as it explores.

It performs the following steps:

- Collects data using cameras and sensors.
- Identifies safe paths and hazardous areas.
- Stores newly discovered information.
- Updates the internal map continuously.
- Replans the route whenever new obstacles are detected.

This allows the rover to improve its navigation and avoid previously discovered hazards.

(iv) Compare Online Search with Uninformed and Informed Search

Online Search	Uninformed Search	Informed Search
Works without a complete map.	Requires a predefined search space.	Uses heuristic information to guide the search.
Learns while exploring.	Explores blindly without guidance.	Finds solutions faster using heuristic knowledge.
Suitable for unknown environments.	Best for known environments.	Best when heuristic information is available.

Online Search is more suitable because Mars is an unknown and continuously changing environment.

(v) Justify the most suitable approach considering uncertainty, adaptability, and safety

Recommended Approach: Online Search Agent

Justification

- Handles unknown environments effectively.
- Adapts to newly discovered obstacles.
- Updates its internal map continuously.
- Makes real-time navigation decisions.
- Ensures safer movement by avoiding hazards.
- Increases the chances of successful sample collection.

4.(i) Clearly define variables, domains, and constraints with explanation

Variables

Each exam is considered a variable that must be scheduled.

Domains

The possible values for each variable are:

- Available dates
- Time slots
- Exam halls

Constraints

- No overlapping exams for the same student.
- Limited number of exam halls.
- Subject order must be maintained where required.
- Faculty must be available for assigned exams.
- Special accommodation requirements must be satisfied.

These constraints ensure that the timetable is valid and conflict-free.

(ii) Explain how Backtracking Search works in this scenario

Backtracking Search assigns a time slot and hall to one exam at a time.

Working:

1. Assign a valid time slot and hall to an exam.
2. Check whether all constraints are satisfied.
3. If no conflict occurs, schedule the next exam.
4. If a conflict is found, go back (backtrack) and try another assignment.
5. Repeat until all exams are successfully scheduled.

Advantages

- Finds a valid timetable if one exists.
- Systematically checks all possibilities.
- Ensures all constraints are satisfied.

(iii) Discuss how Constraint Propagation (Forward Checking) improves efficiency

Forward Checking improves Backtracking by checking future assignments before they are made.

Benefits

- Removes invalid choices early.
- Detects conflicts before they occur.
- Reduces unnecessary backtracking.
- Speeds up timetable generation.
- Improves efficiency for large scheduling problems.

(iv) Analyze real-world challenges and justify the best strategy for scalability

Real-world Challenges

- Large number of students and courses.
- Limited exam halls and faculty members.
- Last-minute schedule changes.
- Special accommodation requirements.
- Increasing complexity as the university grows.

Recommended Strategy : Backtracking Search with Forward Checking (Constraint Propagation)

Justification

- Efficiently handles multiple constraints.
- Reduces the search space.
- Minimizes conflicts and unnecessary backtracking.
- Produces valid exam schedules.
- Scales better for large universities than simple backtracking alone.

5.(i) Explain how the Minimax algorithm models this problem

The Minimax algorithm models the game as a two-player competition:

- **MAX player** – the AI tries to maximize its score.
- **MIN player** – the human opponent tries to minimize the AI's score.

The algorithm explores possible future moves and selects the move that gives the AI the best outcome, assuming the opponent also plays optimally.

Advantages

- Makes logical and strategic decisions.
- Considers both the AI's and the opponent's moves.
- Helps the AI choose the best possible action.

(ii) Analyze the computational challenges of large game trees

In real-time strategy games:

- Every move creates many possible future moves.
- The number of game states grows very quickly.
- Searching the entire game tree requires a large amount of time and memory.
- Real-time decision making becomes difficult because responses must be fast.

These challenges make the standard Minimax algorithm computationally expensive.

(iii) Explain how Alpha-Beta Pruning improves efficiency

Alpha-Beta Pruning is an optimization of the Minimax algorithm.

It works by:

- Eliminating branches that cannot affect the final decision.
- Avoiding unnecessary calculations.
- Reducing the number of nodes explored.
- Producing the same optimal result as Minimax with less computation.

Advantages

- Faster decision making.
- Lower memory usage.
- Suitable for large game trees.
- Improves real-time performance.

(iv) Design an evaluation function and justify the factors considered

The evaluation function estimates how good a game state is for the AI.

Factors considered

- Number of AI units remaining.
- Number of opponent units remaining.
- Health of AI units.
- Resources collected.
- Territory or map control.
- Distance to objectives.
- Defensive and attacking strength.

These factors help the AI compare different game states and choose the most beneficial move.

(v) Recommend a strategy for balancing optimality and real-time performance

Recommended Strategy: Minimax with Alpha-Beta Pruning

Justification

- Produces strong strategic decisions.
- Reduces unnecessary search using Alpha-Beta Pruning.
- Responds quickly within real-time limits.
- Maintains good decision quality while reducing computation.
- Suitable for complex strategy games with large decision trees.